PrivacyTwin
Technical report draft

Find personal data. Keep its evidence current.

> This draft documents the implemented prototype and a fresh synthetic run. Apply the organizer's Proposed-structure-hackathon.pdf before submission. Team details and independently human-labelled evaluation remain to be supplied.

Measured run: 2026-10-29T16:50:19.155+00:00
Frozen detector policy: 72a1cecd0026f75612ef6f3835e5c95fceeebc5ca9d786ba69f4f95baf3991b9
Evaluation input digest: 5d16b2274fa68498b1dac3c0e57389b54763aa718562d62a2acc6d4370f3dc33

1. Problem and implemented scope

Structured files and tables can contain candidate personal-data fields with incomplete purpose and retention documentation. A saved human decision can become unreliable after evidence changes. PrivacyTwin inspects synthetic CSV and SQLite data, produces aggregate technical evidence, links explicit declarations, and records classification and metadata reviews with separate dependencies.

The configured classes are Identity, Contact, Location, Official ID and Demographic. PS-019.pdf requests five predefined classes without enumerating them; the team should confirm this taxonomy with the mentor. The tool makes no automatic lawful-basis determination or universal data-flow inference.

2. Architecture and data handling

Next.js presents Sources → Rules → Inventory → Reviews → Metadata → Changes → Results. FastAPI coordinates a modular Python core. SQLite stores versioned detector policies, immutable aggregate snapshots, explicit declarations, human reviews and append-only events. Sources are controlled local copies; the SQLite adapter reads source tables with read-only access.

Within configured limits, each admitted field is scanned in full. Limits are 10 MB per upload, 20,000 rows per table, 100 columns, 25 SQLite tables and 250,000 cells per source. Exact source/table/column keys determine identity. A rename without a mapping creates a new identity. A failed scan preserves committed evidence.

Raw uploaded files remain in the local source-copy directory; they are not included in aggregate evidence exports. Field names, declarations and reviewer text remain visible in evidence packages. Optional Gemini assistance sends selected aggregate evidence to an external provider; the deterministic scanner, challenge, evaluation and verifier require no model API.

3. Detection and evidence

Configurable rules combine header hints, full-value patterns, normalization and structural validation. Aadhaar-shaped values use Verhoeff validation; GSTIN-shaped values use a check digit. Format/checksum matches are candidates and do not establish issuance, ownership or personal/business status. Declared field meaning is explicit owner context.

Within a class and signal group, only the strongest contribution counts, limiting duplicate-rule inflation. Group contributions are weighted and compared to a configured acceptance threshold. Candidate scores, competing interpretations, mixed evidence and unknown states remain visible. These scores are heuristic evidence weights, not calibrated probabilities.

The sensitivity workbench removes selected signal groups or checksum validation using stored aggregates and the snapshot's frozen scoring policy. Removed weights are not redistributed. Rejected-format counts are retained so checksum ablations can be calculated without rereading raw values. Older snapshots require rescanning for complete validation detail.

4. Metadata and ReviewGuard

Field declarations completely override table defaults; missing properties stay gaps. The configurable checklist covers declared purpose, retention duration/unit, start trigger, owner and provenance. A draft preview calculates gaps and affected review scopes without a write. Saving requires the matching preview and current revisions.

Classification reviews depend on candidate evidence, relevant detector revisions and scoring/materiality policies. Metadata reviews depend on explicit declarations and checklist/resolver policies. Reviews reopen only after the corresponding dependency comparison indicates a material change. Reversion never silently reapproves an already reopened decision.

```
for each exact source / table / column:
    resolve explicit declaration and provenance
    profile values and run configured detectors
    group evidence, calculate candidates and retain uncertainty
    save immutable aggregate snapshot
    for each saved review scope:
        compare original dependencies with current dependencies
        preserve / reopen / mark missing, recording the reason
show missing metadata, require human review, export evidence
```

5. Reproducible synthetic demonstration

The artifact builder creates a disposable workspace, scans actual generated files and leaves the user's workspace untouched. The measured run contains 4 sources and 22 fields. No human approvals are preloaded into this evidence package.

Source	Format	Fields	Rows across tables
customers.csv	CSV	7	50
accounts.csv	CSV	4	30
employees.csv	CSV	5	20
directory.db	SQLite	6	20

The interactive challenge separately creates scripted synthetic reviews through the ordinary Service methods and applies eight controlled changes. Expected review states are specified before observing the result. It checks affected and unrelated reviews, plus the new unreviewed identity after a rename.

Scenario	Matched checks	Expected behavior
Change values, keep the header	4 / 4	Only the classification review reopens.
Remove the declared purpose	4 / 4	Only the metadata review reopens.
Change a relied-on rule	4 / 4	Only the classification review reopens.
Change an unrelated rule	4 / 4	All four reviews stay current.
Rescan identical data	4 / 4	All four reviews stay current.
Rename without a mapping	5 / 5	Old reviews become missing; the new field has no reviews.
Change values, then restore them	4 / 4	Classification stays reopened; metadata stays current.
Stay within the rate tolerance	4 / 4	All four reviews stay current.

33 / 33 states matched; 0 false reopenings and 0 missed reopenings. These are scripted regression results under the reference configuration, not estimates across real-world change distributions.

6. Classification evaluation and baselines

The developer-authored synthetic set has 12 benchmark fields and 7 separate stress fields. Each positive class has support 2 in the benchmark. Precision is TP/(TP+FP); recall is TP/(TP+FN). A positive

field with no predicted class counts as a false negative. Macro F1 averages supported classes. This is single-class field evaluation; inherently multi-class fields require separate annotation and evaluation.

Class	Precision	Recall	Support
Identity	1.000	1.000	2
Contact	1.000	1.000	2
Location	1.000	1.000	2
Official ID	1.000	1.000	2
Demographic	1.000	1.000	2

Method	Benchmark macro F1	Stress errors / cases
Header rules	0.933	4 / 7
Headers + patterns	1.000	2 / 7
Full validation	1.000	0 / 7

Same cases and acceptance threshold. Header-only assigns weight 1 to headers; the pattern variant removes only Aadhaar/GSTIN checksums. Full validation uses the saved configuration. Stress cases stay separate. No improvement is assumed.

> The small, developer-authored fixtures are insufficient to establish general detection accuracy. They are not independently held-out manual annotation. Follow docs/EVALUATION_PROTOCOL.md to obtain the manually labelled evaluation required by PS-019; retain annotation provenance and freeze rules before testing.

7. Portable evidence and verification

The evidence ZIP is produced from one database read transaction. It contains the passport, inventory, gaps, history, current configuration, latest evaluation when available, a file manifest and a standalone standard-library Python verifier. The verifier checks SHA-256 file consistency, policy content identifiers, scan/review references and independent aggregate candidate-score replay.

This generated package replayed 22 snapshots across 8 manifest-listed files. Run `python verify_evidence.py synthetic-evidence.zip` using the included script. Hashes without an external trusted signature do not establish authenticity, correctness of observations or legal validity.

8. Validation, limitations and submission work

Backend and browser suites cover adapters, scoped reviews, stale saves, preview purity, exports, responsive navigation, sensitivity, challenge isolation and package alteration detection. The current test run is recorded separately in VERIFICATION.md to distinguish software regression checks from detection evaluation.

Limits: single-user local deployment; bounded structured input; hand-configured scoring; uncertain person/business linkage; no independent accuracy estimate yet. Optional AI output is explicitly attributed and cannot supply missing owner declarations. The default demo requires no provider availability.

Before submission: complete independent manual annotation; confirm the taxonomy; add team identities; adapt this report to the official template; complete the required private GitHub collaboration/ownership process; rehearse the five-minute script and record a backup demonstration. This draft does not perform external repository administration.

Supporting repository artifacts: docs/PITCH.md; docs/EVALUATION_PROTOCOL.md; core/engine.py; core/service.py; core/challenge.py; core/evaluation.py; tools/verify_evidence.py; tests/test_proof.py; web/tests/proof.spec.ts.